EcoLab1

Создание компонента, реализующего алгоритм сортировки с помощью бинарного дерева (Tree sort)

1. Алгоритм
2. Асимптотика
3. Выбор подхода
4. Функция
5. Результаты тестирования
6. Поддержка других компонентов
7. Визуализация

Работу подготовил

Корнев Егор

20ПИ-1

1. Алгоритм

В задаче требуется реализовать сортировку с помощью двоичного дерева, или же Tree sort. Принцип работы данной сортировки заключается в том, что создается бинарное дерево поиска, элементы сортируемого массива добавляются в дерево в качестве вершин, а далее, с помощью обхода дерева по порядку (in-order tree traversal), можно получить отсортированную последовательность и записать ее в первоначальный массив. Сортируются же элементы благодаря свойствам бинарных деревьев поиска, а именно:

1. Оба поддерева должны быть бинарными деревьями поиска.
2. У всех узлов *левого* поддерева произвольного узла X значения ключей данных *меньше либо равны*, нежели значение ключа данных самого узла X.
3. У всех узлов *правого* поддерева произвольного узла X значения ключей данных *больше*, нежели значение ключа данных самого узла X.

Благодаря им дерево будет отсортировано при обходе in-order.

Также, сортировка не является стабильной, то есть, исходный порядок элементов с одинаковыми ключами не сохраняется.

2. Асимптотика

Добавление объекта в бинарное дерево поиска имеет алгоритмическую сложность $O(\log n)$. Получается, для n элементов в массиве сложность сортировки, в среднем, будет равняться $O(n \log n)$. Но у бинарных деревьев есть такой недостаток: при добавлении постоянно возрастающих (убывающих) элементов в дерево, сложность добавления нового элемента деградирует до $O(n)$, где n – это количество уже имеющихся в дереве элементов. В таком случае сортировка будет составлять уже не $O(n \log n)$, а $O(n^2)$, ведь при добавлении нового элемента нам придется проходить все уже добавленные до этого. Происходит это из-за того, что каждый элемент строго больше (меньше) предыдущих и добавление происходит только в одну сторону дерева: левую или правую. Такое дерево, превратившееся, по сути, в связной список

является критическим случаем, но разбалансировка может происходить в ту или иную сторону и при, казалось бы, обычных входных данных.

Говоря об сложности по памяти – она всегда $O(n)$, ведь мы выделяем n вершин в дереве для сортировки.

3. Выбор подхода

Чтобы исправить проблему с деградацией асимптотики в дереве существует несколько подходов, например, Красно-черные деревья (Red-black trees) или AVL-деревья. Все они призваны при каждом новом элементе балансировать дерево, дабы с левой и с правой стороны было примерно равное количество элементов (всегда равное в случае AVL-деревьев). Происходит это за счет так называемых *вращений дерева*. Одно вращение у обоих видов деревьев имеет алгоритмическую сложность $O(\log n)$.

При разработке мною было замечено, что обычное бинарное дерево поиска действительно крайне медленно обрабатывает кейсы с уже сортированным массивом и иногда возникают проблемы с производительностью на деревьях от 500 тысяч вершин из-за того, что дерево при добавлении получилось несбалансированным. Для улучшения показателей я использовал подход Красно-черного дерева, так как он балансирует дерево, но не гарантирует его *полную сбалансированность*, так, не происходят ненужные дополнительные вращения дерева, как, например, в AVL-деревьях.

Таким образом, коэффициент $\log n$ в $O(n \log n)$ увеличится из-за вращений дерева, но сортировка стабилизируется, так как худший случай с постоянно возрастающими (убывающими) элементами превратится в обычный и не будет критически замедлять работу алгоритма. Сортировка также ускорится на больших входных данных, ведь будет присутствовать балансировка элементов в дереве.

4. Функция

Поскольку сигнатура функции должна быть такой же, как у qsort из стандартной библиотеки Си, то выглядит она так:

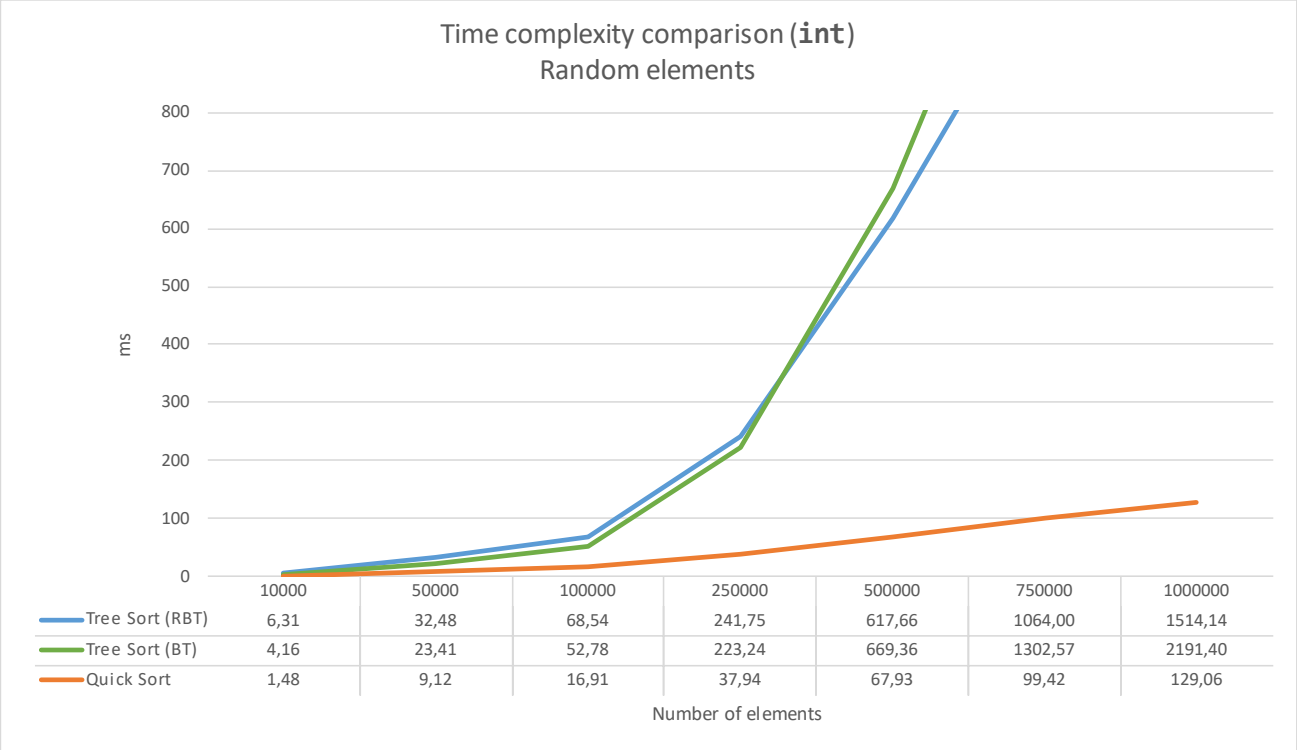
```
int16_t ECOCALLMETHOD CEcoLab1_TreeSort(/* in */ struct IEcoLab1* me,
                                        /* in */ void* p_Base,
                                        /* in */ size_t u32Num,
                                        /* in */ size_t u32Size,
                                        /* in */ int (ECOCDECLMETHOD*
fnCompare)(const void*, const void*))
```

Здесь мы видим параметры, где `p_Base` – это массив, который нужно отсортировать, массив размера `u32Num` с байтовым размером элемента `u32Size`. Будущие сравнения элементов нужно будет производить с помощью передаваемой функции `fnCompare`.

В отличии от qsort данная реализация имеет возвращаемый тип `int16_t`. Он нужен для обработки ошибок. Если сортировка успешна возвращаемым значением будет 0, если нет, то -1.

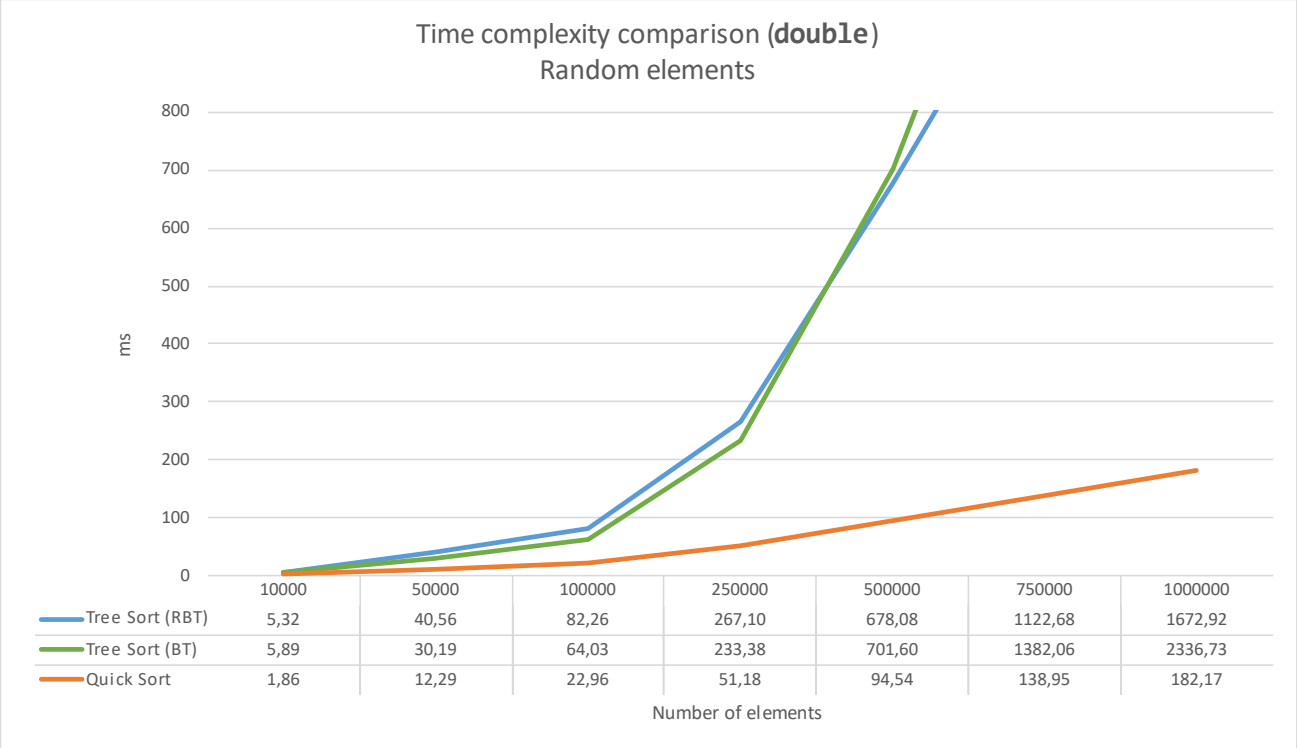
5. Результаты тестирования

Здесь приведены результаты тестирования по времени на случайных элементах типа `int`. Конечно же, древесная сортировка сильно проигрывает быстрой сортировке по времени исполнения и интереснее посмотреть сравнение между двумя подходами – Binary Tree (BT) и Red-black tree (RBT). На длинной дистанции (с 250000 элементов) RBT выполняется чуть быстрее, чем BT. Происходит это из-за того, что при классическом подходе часто может возникнуть ситуация, когда дерево построилось несбалансированным. От этого и происходит увеличение времени выполнения сортировки.

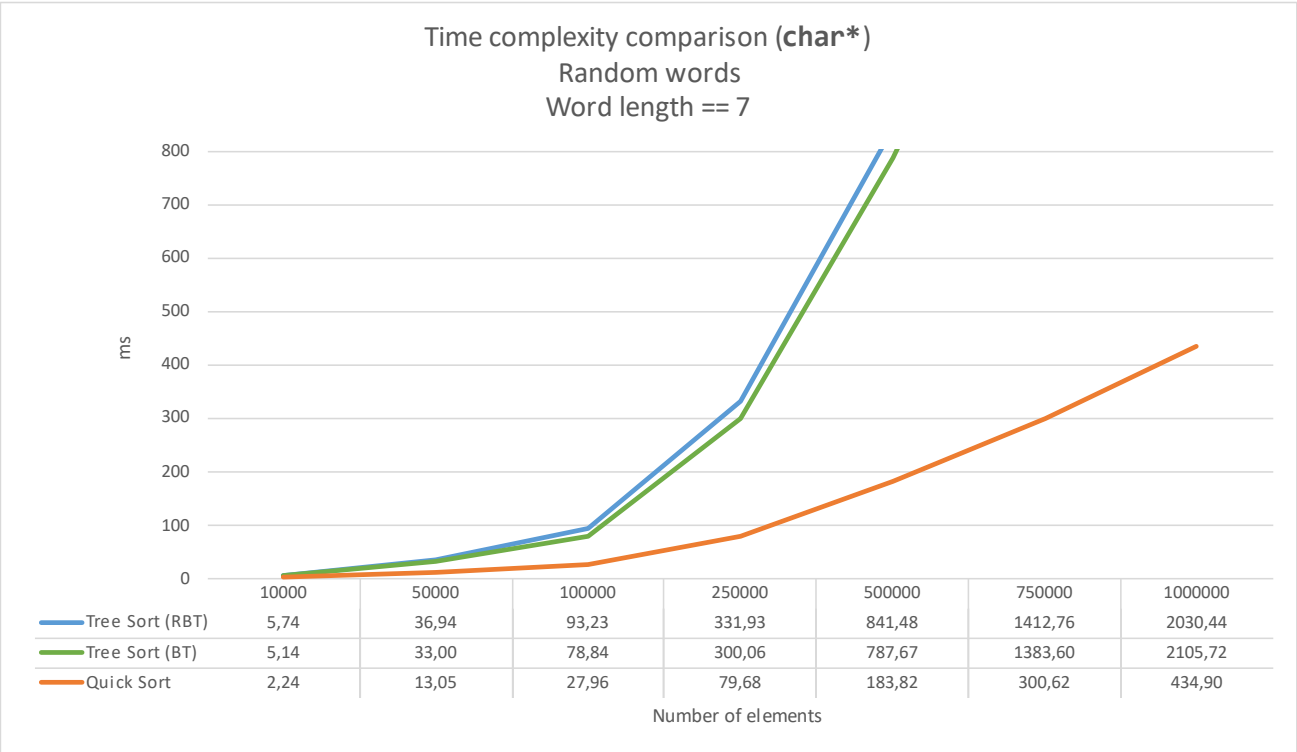


	10000	50000	100000	250000	500000	750000	1000000
Tree Sort (RBT)	6,31	32,48	68,54	241,75	617,66	1064,00	1514,14
Tree Sort (BT)	4,16	23,41	52,78	223,24	669,36	1302,57	2191,40
Quick Sort	1,48	9,12	16,91	37,94	67,93	99,42	129,06

Такой же эффект можно увидеть на тестировании сортировки для случайных чисел типа `double`.

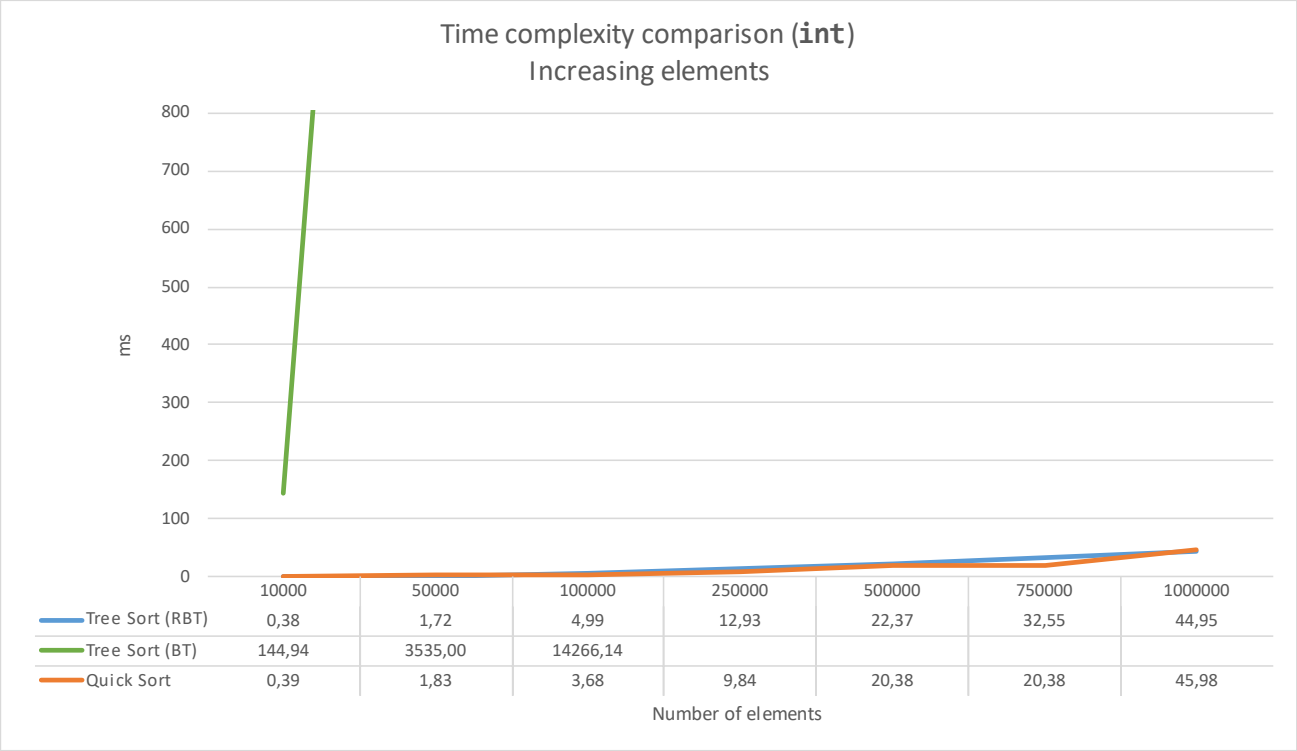


Но, например, в случае слов, то есть `char*` ситуация обратная. Но данная сортировка сильно зависит от предоставленной функции сравнения слов.



В тестировании функции на кейсах с возрастающими элементами типа `int` становится очевидно превосходство подхода RBT над BT. BT подходу потребовалось

14 секунд, чтобы отсортировать 100000 элементов. Данные по последующим тестам отсутствуют, так как время исполнения растет квадратично и окончания исполнения этих тестов просто не дождаться. RBT же справляется с этими тестами иногда даже лучше, чем быстрая сортировка.



6. Поддержка других компонентов

В качестве второй лабораторной работы задача была добавить в реализованный ранее компонент CEcoLab1 поддержку других компонентов через способы агрегирования и включения. Также нужно реализовать взаимозаменяемость компонентов. Тестовыми компонентами для демонстрации были выбраны A, B, E и D, реализующие базовые математические операции.

Для поддержки сторонних компонентов требуется сначала изменить структуру основного – CEcoLab1, а именно добавить туда указатели на новые интерфейсы и их виртуальные таблицы. Выглядит это так:

```
typedef struct CEcoLab1 {

    /* Таблица функций интерфейса IEcoLab1 */
    IEcoLab1VTbl* m_pVTblIEcoLab1;

    /* Указатель для получения интерфейса компонента A методом включения (или B
при неудаче A) */
    IEcoCalculatorX* m_pIEcoCalculatorX;

    /* Указатель для получения интерфейса компонента D методом включения (или E
при неудаче D) */
    IEcoCalculatorY* m_pIEcoCalculatorY;

    IEcoCalculatorXVTbl* m_pVTblIEcoCalculatorX;

    IEcoCalculatorYVTbl* m_pVTblIEcoCalculatorY;

    /* Указатель для получения компонента B методом агрегирования */
    IEcoUnknown* m_pInnerUnknown;

    ...

} CEcoLab1, *CEcoLab1Ptr;
```

Чтобы указателям были присвоены нужные интерфейсы должна пройти инициализация основного компонента в методе `initCEcoLab1`, где дополнительно будут инициализированы указатели.

Сначала продемонстрируем метод включения, а также взаимозаменяемость компонентов на примере D/E.

```
result = pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoCalculatorD, 0,
&IID_IEcoCalculatorY, (void**) &pCMe->m_pIEcoCalculatorY);
    if (result != 0 || pCMe->m_pIEcoCalculatorY == 0) {
        result = pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoCalculatorE, 0,
&IID_IEcoCalculatorY, (void**) &pCMe->m_pIEcoCalculatorY);
}
```

Как видно, указатель на интерфейс `IEcoCalculatorY` переходит к члену структуры CEcoLab1. Если по какой-то причине не выходит получить интерфейс компонента D, то запрашивается такой же интерфейс только уже от компонента E.
В методе включения наш компонент CEcoLab1 содержит указатель на интерфейс `IEcoCalculatorY` и использует его, получается, для того чтобы конечному пользователю получить доступ к внутреннему компоненту нужно дополнительно имплементировать методы `IEcoCalculatorY` во внешнем компоненте. Внутренний

компонент содержит два метода – Multiplication и Division. Вот пример имплементации одного из них во внешнем компоненте:

```
int32_t CEcoLab1_Multiplication(/* in */ struct IEcoCalculatorY* me, /* in */
int16_t a, /* in */ int16_t b) {
    CEcoLab1* pCMe = (CEcoLab1*) ((uint64_t) me - sizeof(struct IEcoLab1*) * 2);

    if (me == 0 || pCMe->m_pIEcoCalculatorY == 0) {
        return -1;
    }

    return pCMe->m_pIEcoCalculatorY->pVTbl->Multiplication(pCMe-
>m_pIEcoCalculatorY, a, b);
}
```

Такая же имплементация будет у метода Division.

Добавим методы в отдельную виртуальную таблицу `IEcoCalculatorXVTbl`. Для доступа пользователя к функциям модифицируем `CEcoLab1_QueryInterface` для того, чтобы он мог возвращать ссылку на виртуальную таблицу по запросу. Вот так:

```
else if (IsEqualUGUID(riid, &IID_IEcoCalculatorY)) {
        *ppv = &pCMe->m_pVTblIEcoCalculatorY;
        pCMe->m_pVTblIEcoLab1->AddRef((IEcoLab1*) pCMe);
}
```

Далее идет метод агрегирования на примере компонента B, который поддерживает этот способ. Агрегирование – это особый вид включения, при котором указатель на интерфейс внутреннего компонента передается пользователю. Для этого способа в структуре CEcoLab1 был создан указатель `m_pInnerUnknown`. Он имеет тип `IEcoUnknown`, такой тип является базовым для интерфейсов и реализует кроме прочих метод `QueryInterface`, который нам понадобится в будущем для передачи указателя пользователю. Запрос компонента выглядит так:

```
result = pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoCalculatorB, pOuterUnknown,
&IID_IEcoUnknown, (void**) &pCMe->m_pInnerUnknown);
```

Поскольку теперь нам нужно передать указатель на интерфейс, то воспользуемся методом `CEcoLab1_QueryInterface` внутри нашего компонента и модифицируем его, добавив ветку `else if`. В ней, если пользователь запрашивает компонент, который мы поддерживаем и указатель на него не равен 0, то происходит вызов метода `QueryInterface` у полученного компонента в указателе `m_pInnerUnknown`. Мы

передаем туда m_pInnerUnknown, id интерфейса и указатель, куда будет возвращен требуемый интерфейс. В коде else if выглядит так:

```
else if (IsEqualUGUID(riid, &IID_IEcoCalculatorX) && pCMe->m_pInnerUnknown != 0)
{
    return pCMe->m_pInnerUnknown->pVTbl->QueryInterface(pCMe->m_pInnerUnknown,
riid, ppv);
}
```

7. Визуализация

В третьей работе нам требуется использовать модель «Издатель-Подписчик» в нашем компоненте. В такой модели «Издатель» публикует какое-либо событие, а «Подписчик» это событие использует. Применение этой модели можно найти в задаче визуализации процесса сортировки. Если проще, то функция сортировки – это «Издатель» и она будет отправлять некоторые события к «Подписчикам», которые в свою очередь будут, например, отображать процесс смены элементов, движение элемента по бинарному дереву и т.д.

Для начала определим, о каких событиях «Издатель» в виде функции TreeSort будет сообщать «Подписчикам». Я определил 4 события или обратных вызова, которые нам понадобятся. Определяются они в файле IEcoLab1Events.h компонента.

```
/* IEcoLab1Events */
int16_t(ECOCALLMETHOD* OnTreePrintCallback)(/* in */ struct IEcoLab1Events* me,
/* in */ struct RBNode* p_Root, /* in */ int16_t bClear);

int16_t(ECOCALLMETHOD* OnTreePrintWithHighlightCallback)(/* in */ struct
IEcoLab1Events* me, /* in */ struct RBNode* p_Root, /* in */ struct RBNode*
p_Highlight, /* in */ int16_t bSecondaryColor, /* in */ void* p_Element);

int16_t(ECOCALLMETHOD* OnTreeInorderTraversalCallback)(/* in */ struct
IEcoLab1Events* me, /* in */ struct RBNode* p_Root);

int16_t(ECOCALLMETHOD* OnSaveCursorPositionCallback)(/* in */ struct
IEcoLab1Events* me);
```

Имплементации callback'ов будут в файле CEcoLab1Sink.c уже клиента, а их вызов сортировка внутри компонента произведет через специальную функцию Fire_*название обратного вызова*, которая пошлет «Событие» всем «Подписчикам».

Теперь опишем суть обратных вызовов для сортировки:

`OnTreePrintCallback` – печатает бинарное дерево

`OnTreePrintWithHighlightCallback` – выполняет ту же функцию, что и `OnTreePrintCallback`, но может «подсветить» выбранную вершину двумя цветами. Каким именно цветом определяет параметр `bSecondaryColor`. Что значит «подсветить» будет объяснено позже.

`OnTreeInorderTraversalCallback` – выполнят обход дерева in-order и показывает процесс с помощью все той же «подсветки» вершин.

Последний вызов – это `OnSaveCursorPositionCallback`. Через него сортировка сообщает, что нужно сохранить текущую позицию курсора в терминале, чтобы на нее потом вернуться. Тут стоит сказать, что это реализовано через так называемые ANSI escape codes. Они позволяют курсору перемещаться по терминалу, очищать его и печатать цветной текст, то есть «подсвечивать» определенные элементы, например. В моей визуализации сортировки они широко используются для демонстрации пути, который проделал элемент, чтобы встать на позицию в дереве, отображения цветов вершин (ведь используются Красно-черные деревья) и очищения терминала, чтобы показать следующие итерации сортировки.

Чтобы добавить в визуализацию элемент интерактивности я решил отказаться от использования функции `sleep()` в коде, потому что она не дает пользователю контроля над процессом сортировки и в моменте достаточно сложно понять, что в алгоритме сейчас происходит. Для исправления этой проблемы я решил реализовать callback'и и их вызовы так, чтобы они пошагово показывали процесс сортировки, а пользователь контролировал этот процесс с помощью какой-либо клавиши. Для этого нам понадобится функция `getch()`, но она не является стандартной и в зависимости от ОС находится в разных заголовочных файлах. Чтобы итоговые тесты работали на обоих системах, которые у меня есть (Windows, Linux) я реализовал адаптер для `getch()`, который устраняет этот недостаток.

Адаптер выглядит так:

```
int getch_(void) {
#ifdef WIN32
    return _getch();
#else
    int ch;
    struct termios oldt, newt;
    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);
    ch = getchar();
    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
    return ch;
#endif
}
```

Различные заголовочные файлы также подключаются через директивы препроцессора `#ifdef` и др.

```
#ifdef WIN32
#include <windows.h>
#include <conio.h>
#else
#include <unistd.h>
#include <termios.h>
#endif
```

Далее реализуем макрос для остановки функции и ожидания ввода пользователя и будем вставлять его там, где нужно сделать паузу и ждать реакции клиента.

```
#define NEXT_KEY 'n'
#define WAIT_FOR_NEXT                   \
do {                                    \
    int c = getch_();                   \
    while (c != NEXT_KEY && c != '\r'   \
           && c != '\n' && c != EOF) {  \
        c = getch_();                   \
    }                                   \
} while (0)
```

Итого, теперь наша функция сортировки поддерживает все необходимые для визуализации callback'и, а также использует пользовательский ввод для контроля за шагами сортировки, причем пользовательский ввод работает как на Linux, так и на Windows.